

Assignment No -5

Insert code cell below (Ctrl+M B) re . Roll No - EN23107080 . Class - TY(B) . Batch - A

Title - Business Model Development for AI Products Create a Business Model Canvas for an AI-based product. Define the value proposition, customer segments, revenue streams, and cost structures. **Ethical Challenges in AI** Analyze a real-world AI application (e.g., facial recognition, deep fake technology, AI bias in hiring) and discuss the ethical challenges involved. Propose solutions to mitigate risks and ensure responsible AI usage.

1) Sample dataframe -

```
experience = [2,5,3,7,1,6,4,8]
gender      = [0,0,0,0,1,1,1,1] # 0 = Female, 1 = Male
hired       = [1,1,1,1,0,0,0,0] # 0 = Rejected, 1 = Hired
```

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
```

```
experience = np.array([2,5,3,7,1,6,4,8])
gender     = np.array([0,0,0,0,1,1,1,1])
hired      = np.array([1,1,1,1,0,0,0,0])
```

```
X = np.column_stack((experience, gender))
y = hired
```

```
model = LogisticRegression()
model.fit(X, y)
```

▼ LogisticRegression ⓘ ?

```
LogisticRegression()
```

```
test_candidate = np.array([[5, 0]])
prediction = model.predict(test_candidate)

print("Prediction (1=Hired, 0=Rejected):", prediction[0])
```

```
Prediction (1=Hired, 0=Rejected): 1
```

```
print("Model Coefficients:", model.coef_)
print("Intercept:", model.intercept_)

Model Coefficients: [[-0.06529192 -1.34180685]]
Intercept: [0.96369527]
```

```
test_male = np.array([[10, 1]])
print(model.predict(test_male))
```

```
[0]
```

2) Dataset -

```
df = pd.read_csv("HR_Analytics.csv")
```

```
df
```

	EmpID	Age	AgeGroup	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	...	
	0	RM297	18	18-25	Yes	Travel_Rarely	230	Research & Development	3	3	Life Sciences	..
Insert code cell below (Ctrl+M B)	1	RM302	18	18-25	No	Travel_Rarely	812	Sales	10	3	Medical	..
	2	RM458	18	18-25	Yes	Travel_Frequently	1306	Sales	5	3	Marketing	..
	3	RM728	18	18-25	No	Non-Travel	287	Research & Development	5	2	Life Sciences	..
	4	RM829	18	18-25	Yes	Non-Travel	247	Research & Development	8	1	Medical	..
...	...	...	...	...	...	...	...	...	...	...	..	
1475	RM412	60	55+	No	Travel_Rarely	422	Research & Development	7	3	Life Sciences	..	
1476	RM428	60	55+	No	Travel_Frequently	1499	Sales	28	3	Marketing	..	
1477	RM537	60	55+	No	Travel_Rarely	1179	Sales	16	4	Marketing	..	
1478	RM880	60	55+	No	Travel_Rarely	696	Sales	7	4	Marketing	..	
1479	RM1210	60	55+	No	Travel_Rarely	370	Research & Development	1	4	Medical	..	
1480 rows x 38 columns												

df.shape

(1480, 38)

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1480 entries, 0 to 1479
Data columns (total 38 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   EmpID                                1480 non-null   object
1   Age                                  1480 non-null   int64
2   AgeGroup                             1480 non-null   object
3   Attrition                            1480 non-null   object
4   BusinessTravel                       1480 non-null   object
5   DailyRate                            1480 non-null   int64
6   Department                           1480 non-null   object
7   DistanceFromHome                     1480 non-null   int64
8   Education                             1480 non-null   int64
9   EducationField                       1480 non-null   object
10  EmployeeCount                        1480 non-null   int64
11  EmployeeNumber                       1480 non-null   int64
12  EnvironmentSatisfaction               1480 non-null   int64
13  Gender                               1480 non-null   object
14  HourlyRate                           1480 non-null   int64
15  JobInvolvement                       1480 non-null   int64
16  JobLevel                             1480 non-null   int64
17  JobRole                              1480 non-null   object
18  JobSatisfaction                      1480 non-null   int64
19  MaritalStatus                        1480 non-null   object
20  MonthlyIncome                        1480 non-null   int64
21  SalarySlab                           1480 non-null   object
22  MonthlyRate                          1480 non-null   int64
23  NumCompaniesWorked                  1480 non-null   int64
24  Over18                              1480 non-null   object
25  OverTime                             1480 non-null   object
26  PercentSalaryHike                   1480 non-null   int64
27  PerformanceRating                   1480 non-null   int64
28  RelationshipSatisfaction              1480 non-null   int64
29  StandardHours                       1480 non-null   int64
30  StockOptionLevel                    1480 non-null   int64
31  TotalWorkingYears                   1480 non-null   int64
32  TrainingTimesLastYear                1480 non-null   int64
33  WorkLifeBalance                     1480 non-null   int64
34  YearsAtCompany                      1480 non-null   int64
35  YearsInCurrentRole                   1480 non-null   int64
36  YearsSinceLastPromotion              1480 non-null   int64
37  YearsWithCurrManager                 1423 non-null   float64
dtypes: float64(1), int64(25), object(12)
memory usage: 439.5+ KB
```

df.describe()

	Age	DailyRate	DistanceFromHome	Education	EmployeeCount	EmployeeNumber	EnvironmentSatisfaction	HourlyRate
count	1480.000000	1480.000000	1480.000000	1480.000000	1480.0	1480.000000	1480.000000	1480.000000
mean	30.917500	801.384459	9.220270	2.910811	1.0	1031.860811	2.724324	65.84527
std	9.128559	403.126988	8.131201	1.023796	0.0	605.955046	1.092579	20.32826
min	18.000000	102.000000	1.000000	1.000000	1.0	1.000000	1.000000	30.00000
25%	30.000000	465.000000	2.000000	2.000000	1.0	493.750000	2.000000	48.00000
50%	36.000000	800.000000	7.000000	3.000000	1.0	1027.500000	3.000000	66.00000
75%	43.000000	1157.000000	14.000000	4.000000	1.0	1568.250000	4.000000	83.00000
max	60.000000	1499.000000	29.000000	5.000000	1.0	2068.000000	4.000000	100.00000

8 rows × 26 columns

df.isnull().sum()

	0
EmpID	0
Age	0
Insert code cell below (Ctrl+M B)	0
Attrition	0
BusinessTravel	0
DailyRate	0
Department	0
DistanceFromHome	0
Education	0
EducationField	0
EmployeeCount	0
EmployeeNumber	0
EnvironmentSatisfaction	0
Gender	0
HourlyRate	0
JobInvolvement	0
JobLevel	0
JobRole	0
JobSatisfaction	0
MaritalStatus	0
MonthlyIncome	0
SalarySlab	0
MonthlyRate	0
NumCompaniesWorked	0
Over18	0
OverTime	0
PercentSalaryHike	0
PerformanceRating	0
RelationshipSatisfaction	0
StandardHours	0
StockOptionLevel	0
TotalWorkingYears	0
TrainingTimesLastYear	0
WorkLifeBalance	0
YearsAtCompany	0
YearsInCurrentRole	0
YearsSinceLastPromotion	0
YearsWithCurrManager	57

dtype: int64

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
# for Attrition
df['Attrition'] = le.fit_transform(df['Attrition'])
print(df['Attrition'].head())

# for Gender
df['Gender'] = le.fit_transform(df['Gender'])
print(df['Gender'].head())
```

```
0    1
1    0
2    1
3    0
4    1
Name: Attrition, dtype: int64
0    1
1    0
2    1
3    1
4    1
Name: Gender, dtype: int64
```

```
df.drop(['EmpID', 'AgeGroup', 'YearsWithCurrManager', 'BusinessTravel', 'Department', 'EducationField', 'JobRole', 'MaritalStatus', 'Sa
```

df

	Age	Attrition	DailyRate	DistanceFromHome	Education	EmployeeCount	EmployeeNumber	EnvironmentSatisfaction	Gender	Hou
0	18	1	230	3	3	1	405	3	1	
1	18	0	812	10	3	1	411	4	0	
2	18	1	1306	5	3	1	614	2	1	
3	18	0	287	5	2	1	1012	2	1	
4	18	1	247	8	1	1	1156	3	1	
...	...	...	...	...	...	...	...	...	...	...
1475	60	0	422	7	3	1	549	1	0	
1476	60	0	1499	28	3	1	573	3	0	
1477	60	0	1179	16	4	1	732	1	1	
1478	60	0	696	7	4	1	1233	2	1	
1479	60	0	370	1	4	1	1697	3	1	

1480 rows × 27 columns

```
df.shape
```

(1480, 27)

```
df.isnull().sum()
```

	0
Age	0
Attrition	0
DistanceFromHome	0
Education	0
EmployeeCount	0
EmployeeNumber	0
EnvironmentSatisfaction	0
Gender	0
HourlyRate	0
JobInvolvement	0
JobLevel	0
JobSatisfaction	0
MonthlyIncome	0
MonthlyRate	0
NumCompaniesWorked	0
PercentSalaryHike	0
PerformanceRating	0
RelationshipSatisfaction	0
StandardHours	0
StockOptionLevel	0
TotalWorkingYears	0
TrainingTimesLastYear	0
WorkLifeBalance	0
YearsAtCompany	0
YearsInCurrentRole	0
YearsSinceLastPromotion	0

dtype: int64

```
print(df['Gender'].value_counts())
```

```
Gender
1    889
0    591
Name: count, dtype: int64
```

```
df['Gender'].value_counts().plot(kind='bar')
plt.title("Gender Distribution")
plt.show()
```

Gender Distribution

800

Insert code cell below (Ctrl+M B)

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
```

```
df['Gender'] = df['Gender']
df['Attrition'] = df['Attrition']
```

```
X = df[['TotalWorkingYears', 'Gender']]
y = df['Attrition']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Gender

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

▼ LogisticRegression ⓘ ?

LogisticRegression()

```
y_pred = model.predict(X_test)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Confusion Matrix:

```
[[249  0]
 [ 47  0]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.84	1.00	0.91	249
1	0.00	0.00	0.00	47
accuracy			0.84	296
macro avg	0.42	0.50	0.46	296
weighted avg	0.71	0.84	0.77	296

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for
classes with no predicted samples. The F-score and F1-score used in
classification_report and the confusion_matrix are still valid.
```